SKED for Claude Cowork

Technical Architecture and User Manual

Socratic Knowledge Elicitation & Documentation

Plugin author	Kostiantyn Kharchenko
Author identifier	0000-0002-7334-8038
Target environment	Claude Cowork
Plugin version	0.1.0
Documentation edition	1.1, 29 October 2026
Inspected package	`Sked-0.1.0-v1.zip`
Repository	https://github.com/konst1970/SKED

Scope of this manual

This document explains how an expert uses SKED in Claude Cowork and how the plugin implements persistent dialogue, knowledge synthesis, document planning, LaTeX generation, and document acceptance. It describes the inspected source package rather than an assumed future implementation. Programmatic checks, model instructions, and human decisions are distinguished throughout.

Source-based documentation with targeted execution checks. A complete interactive Cowork session was not executed for this edition.

Abstract

SKED is a Claude plugin for producing structured technical documents through a dialogue with a human expert. Its central design decision is to represent the developing knowledge explicitly before generating the final prose. The plugin combines six skills, a document reviewer agent, three Python utilities, reference instructions, and a LaTeX report class. A five-phase workflow separates initialization, Socratic dialogue, synthesis of an agreed knowledge space, document structuring, and production with quality assessment. An append-only event ledger records claims, definitions, assumptions, decisions, requirements, questions, and sources. A Jaccard-based indicator measures changes between successive ledger states. File hashes bind approval records to particular consensus and outline files. A composite acceptance score combines tagged knowledge coverage, a coherence assessment, and formal checks. This manual documents the user interaction, component architecture, data model, commands, mathematical definitions, build pipeline, and recovery procedures. It also identifies the limits of the implementation. Ledger stability does not establish truth, tags do not establish semantic completeness, and recorded verification remains dependent on the reviewer. The theoretical account is reconstructed from the material bundled with the plugin because the separately mentioned SKED v1.0.0 report is unavailable. Targeted execution checks support selected implementation statements without implying full Cowork compatibility or empirical validation of the method.

Keywords knowledge elicitation; Socratic dialogue; Claude Cowork; plugin architecture; provenance; technical documentation; LaTeX; human review.

Reading guide and evidence conventions

Sections 1–3 introduce the system and its use in Cowork. Sections 4–8 describe the architecture, representation, phases, convergence, and quality mechanisms. Sections 9–11 cover rendering, commands, and an illustrative workflow. Sections 12–14 describe limitations, recovery, and verification. The appendices provide source traceability and reference material.

Implemented identifies behavior established by inspection of executable code. *Prescribed* identifies behavior requested by a skill or reference instruction and executed by Claude. *Recommended* identifies operational advice added in this manual. *Verified in a targeted check* identifies a behavior exercised outside Cowork. An explanation reconstructed from the package is an interpretation of this implementation, not a claim that an external theory has been independently validated.

Contents

Abstract **1**

1 Purpose and system boundary **4**

1.1 The problem addressed 4

1.2 What the plugin contains 4

1.3 Appropriate use 4

2 Theoretical and methodological basis **4**

2.1 Knowledge elicitation as a dialogue 4

2.2 Four Socratic operations 5

2.3 Question selection and expert control 5

2.4 Context engineering 6

2.5 Provenance, replay, and staged commitment . . . 6

3 Using SKED in Claude Cowork 6
3.1 Installation and environment preparation . . . 6
3.2 Starting a session . . . 7
3.3 Participating in the dialogue . . . 7
3.4 Stopping, approving, and receiving output . . . 7
3.5 Continuing and revising . . . 8

4 Component architecture 8
4.1 Layered organization . . . 8
4.2 Skills and reviewer . . . 8
4.3 Executable utilities . . . 9
4.4 Execution and persistence boundary . . . 9

5 Session artifacts and knowledge representation 10
5.1 Directory structure . . . 10
5.2 Entries and provenance . . . 10
5.3 Event operations and replay . . . 11
5.4 Consensus export . . . 11

6 Five phases and transition checks 11
6.1 Initialization . . . 11
6.2 Socratic dialogue . . . 12
6.3 Synthesis . . . 12
6.4 Structuring . . . 12
6.5 Production and acceptance . . . 12
6.6 What transition checks guarantee . . . 12

7 Convergence and dialogue indicators 13
7.1 The state projection . . . 13
7.2 Interpretation and an example . . . 14
7.3 Default stopping conditions . . . 14
7.4 Model-proposal acceptance warning . . . 14

8 Document quality and acceptance 14
8.1 Composite score . . . 14
8.2 Tagged knowledge coverage . . . 15
8.3 Coherence assessment . . . 16
8.4 Formal compliance . . . 16
8.5 Hard blockers and procedural failures . . . 16

9 LaTeX production and bibliography pipeline 17
9.1 Scaffolding and content generation . . . 17

9.2 Profiles and their scope . . . 17
9.3 Bibliography rendering . . . 18
9.4 Build and inspection . . . 18

10 Command and configuration reference 18
10.1 Execution convention . . . 18
10.2 Session commands . . . 19
10.3 Default parameters . . . 19
10.4 Build commands . . . 20

11 Illustrative workflow 20
11.1 Scenario and initialization . . . 20
11.2 Elicitation and ledger changes . . . 20
11.3 Consensus and structure . . . 21
11.4 Production and delivery . . . 21

12 Limitations and assurance boundary 21
12.1 Knowledge and model limitations . . . 21
12.2 Approval and state consistency . . . 22
12.3 Coverage and quality limitations . . . 22
12.4 Persistence, compatibility, and portability . . . 22
12.5 Data handling and scope of protection . . . 22

13 Troubleshooting and recovery 23

14 Verification performed for this edition 23
14.1 Inspection scope . . . 23
14.2 Targeted execution checks . . . 24
14.3 Recommended acceptance exercise . . . 24

A Source traceability 24

B Reference material and document maintenance 25

C Terminology 25

1 Purpose and system boundary

1.1 The problem addressed

A human expert may understand a procedure or system without possessing an explicit, complete description of it. Important knowledge can be distributed across examples, informal terminology, exceptions, and operational decisions. A language model can generate fluent text before those matters have been resolved. SKED addresses this situation by making elicitation and agreement explicit activities that precede document production.

The user contributes domain knowledge, decides the intended scope, challenges proposed interpretations, and approves the resulting knowledge and structure. Claude asks questions, extracts atomic statements, maintains the session artifacts, proposes synthesis, and prepares the report. Python utilities retain state and compute checks. XeLaTeX and Poppler provide compilation and inspection artifacts. These responsibilities are connected, but they are not interchangeable. In particular, executing a command that records approval is different from obtaining the expert's informed approval.

1.2 What the plugin contains

The package is a plugin named `sked`, version `0.1.0`. It is not a single skill. Six skills divide the work by phase, and one agent defines a separate coherence-review role. Three Python scripts implement state management, report building, and bibliography rendering. Five reference files explain method behavior, ledger representation, Socratic operations, profiles, and LaTeX usage. Two template files provide the report class and initial document.

The inspected manifest declares the author and descriptive metadata. The archive contains no connector configuration, MCP server configuration, hooks, marketplace manifest, or dedicated `commands/` directory. Web research is requested through the host environment's available tools rather than through a research service bundled with SKED. The plugin also contains no model weights or training procedure.

1.3 Appropriate use

SKED is intended for technical reports, method descriptions, research documentation, and other documents whose correctness depends on expert knowledge. It is particularly relevant when the author needs traceability between agreed statements and final sections, when assumptions need to remain distinguishable from facts, or when the document will be revised across multiple sessions.

The dialogue adds effort. The starting skill therefore prescribes checking whether a short request could be handled directly. A brief paragraph with all facts already provided may not benefit from a full session. SKED does not replace experiments, independent source evaluation, domain validation, or the publication requirements of a particular journal.

Implementation reference .claude-plugin/plugin.json;skills/sked-start/SKILL.md;README.md.

2 Theoretical and methodological basis

2.1 Knowledge elicitation as a dialogue

The package describes an expert H, a model M, a domain D, and their respective knowledge K_H and K_M. The conceptual goal of the dialogue is an agreed knowledge space

$$K^* = f(K_H, K_M, D_{\text{dialogue}}). \tag{1}$$

The symbols have the following meanings.

- K^* is the consensus knowledge space approved by the human expert. The superscript $*$ denotes the agreed result and does not denote a mathematical optimum.
- K_H is the domain knowledge contributed by the human expert H; the subscript H identifies its human origin.
- K_M is the domain knowledge or proposed interpretation contributed by the language model M; the subscript M identifies its model origin.
- D_{dialogue} is the ordered sequence of questions, answers, challenges, and clarifications exchanged during elicitation. Here D_{dialogue} denotes dialogue data, whereas the standalone symbol D in the surrounding conceptual model denotes the subject domain.
- f denotes the conceptual synthesis process that combines these contributions through elicitation, revision, conflict resolution, and approval. It is not a fitted numerical function.

The expression is a conceptual description. The implementation does not estimate a mathematical function f from data. It operationalizes the expression through questions, ledger changes, conflict resolution, and human approval.

The document is subsequently represented as

$$R = g(K^*, T_{\text{structure}}, S_{\text{style}}), \tag{2}$$

The symbols have the following meanings.

- R is the resulting technical document, including its textual and planned nontextual content.
- g denotes the document-production process, including content realization and presentation according to the agreed plan.
- K^* is the human-approved consensus knowledge space that supplies the document's substantive content.
- $T_{\text{structure}}$ is the approved document structure, including sections, subsections, planned artifacts, and their knowledge-entry assignments. The subscript structure is a descriptive label.
- S_{style} is the selected presentation profile and applicable formatting requirements. The subscript style is a descriptive label.

These objects and mappings are conceptual rather than physical quantities and have no physical units. This separation makes the intended knowledge basis explicit before surface realization. It does not establish that the generated document necessarily preserves every meaning in that basis.

2.2 Four Socratic operations

The reference instructions associate four operations with concrete dialogue behavior. *Maieutics* elicits implicit expert knowledge through questions and examples. An answer may become one or more ledger entries attributed to the human. *Elenchus* tests a claim through a counterexample, boundary condition, or consistency check. It operates in both directions because the human is invited to challenge the model's proposals. *Aporia* represents acknowledged uncertainty as an open question. *Dialectic* proposes a synthesis when positions conflict, leaving the decision to the human.

These terms organize the interaction policy. No formal logical prover or truth-maintenance system is included. A contradiction becomes visible when Claude identifies it and records a contested status. The Python engine does not infer contradictions from natural language.

2.3 Question selection and expert control

The prescribed selection order prioritizes contested entries, open high-priority questions, pending model proposals, untested high-priority human claims, and structural gaps. If these are exhausted, Claude asks what a newcomer might still misunderstand. This ordering is intended to direct attention toward unresolved knowledge rather than premature prose.

In deep mode, the skills request one question per turn. In brisk mode, they allow up to three related questions. The reference also asks Claude to invite challenges to its framing every third round and after contributing content. These are model instructions. The command-line engine stores the mode but does not enforce the number, wording, or intellectual quality of questions.

2.4 Context engineering

Initialization collects a context packet $\{A, E, C, R, M\}$. In this notation, A describes authority and audience, E contains exemplars, C defines constraints, R records the rubric, and M contains document metadata. The letter M here denotes metadata rather than the model used in the earlier conceptual account. In implementation terms, these values are stored under `context` in `session.json`.

Context entries explain what document should be produced and for whom. Constraints can include language, length, source rules, or institutional formatting. The rubric influences the intended assessment priorities. Exemplars guide interpretation and style but should not silently replace the author's claims. The starting instructions classify facts extracted from user-provided materials as human-origin contributions. That classification records provenance, not independent verification.

2.5 Provenance, replay, and staged commitment

The ledger represents knowledge as identifiable entries with origins, statuses, priorities, and source links. Changes are appended as events. The current state is reconstructed by replaying those events. This is an event-sourced representation at the application level. It retains previous revisions and supports inspection of historical rounds, but it has no cryptographic event chain or tamper-resistant storage.

Approval of the consensus and outline creates staged commitments. The engine hashes each approved file and checks that hash at later phase transitions. This detects changes to those files. It does not prove who approved them, establish their factual correctness, or bind their contents to every future ledger update.

The package's reference file names a separate technical report, SKED v1.0.0. That report is not available and is not part of the inspected archive. Accordingly, this manual treats the bundled reference and executable implementation as its method specification. It makes no inherited empirical claims from the unavailable report.

Implementation reference references/sked-method.md;references/socratic-moves.md.

3 Using SKED in Claude Cowork

3.1 Installation and environment preparation

Anthropic documents custom plugin upload through the Claude plugin interface. Open Cowork, then Customize and Plugins, and use the custom-file upload control available in the installed client. Select the SKED archive and confirm that the installed plugin is named `sked`. Exact button labels can vary across client versions. The official plugin guide is listed in Appendix B. The archive's acceptance in a particular Cowork client was not exercised for this manual.

Choose a dedicated working folder and grant the Cowork task access to it. Keep background materials in that folder or provide them as task inputs. Ask Claude to verify that it can read the plugin resources, create files in the selected working folder, and execute Python. The scripts also require an accessible XeLaTeX toolchain for PDF generation. Package installation, outbound research access, and subprocess execution depend on the Cowork environment.

A useful readiness request is the following.

```
Check that the SKED plugin is available in this Cowork task.
```

```
Locate its scripts and templates. Check Python, XeLaTeX,
Poppler, fonts, and write access to the selected folder.
Report missing prerequisites before creating a session.
```

This request is an operational recommendation rather than a packaged SKED command. It prevents an installation success from being mistaken for a successful document build.

3.2 Starting a session

Start in ordinary language. State the subject, audience, output language, and desired document type. Provide formatting requirements and useful source material immediately if they are already known. For example,

```
Start SKED for an English technical report about our dataset
cleaning procedure. The audience is research engineers.
Use the IEEE report profile. Ask questions in Ukrainian.
Record uncertainties explicitly and do not invent measurements.
```

The starting skill creates the session, collects missing context, proposes an initial frame of approximately six to fifteen entries, asks the first question, closes round zero, and moves to the dialogue phase. The initial frame is a set of model proposals to be tested. It should not be treated as the expert's position merely because it was generated first.

The plugin instructions use `CLAUDE_PLUGIN_ROOT` to locate packaged resources. If the host does not expose that value as expected, Claude must locate the actual installed plugin directory before invoking scripts. The path is an execution detail, not a user requirement to place the plugin in a particular host folder.

3.3 Participating in the dialogue

Answer from your own knowledge and distinguish established facts from estimates, preferences, and unresolved questions. Give boundary cases and examples when possible. Correct a recorded entry if Claude has altered the meaning. Ask to see individual entries or the full current ledger whenever the summary is insufficient.

The skills prescribe a compact progress summary after each turn, including the round, ledger changes, and unresolved questions. A productive correction might say that a claim applies only to a particular input size, that an example is illustrative, or that a number has not been measured. Claude should update the corresponding entry rather than merely acknowledge the correction in conversational prose.

When Claude contributes a factual claim, request the supporting source if it matters to the document. Agreeing that an interpretation is useful does not establish that its numerical or bibliographic details are correct. A source-marked entry records that a verification step was reported, so the source itself should still be inspectable.

3.4 Stopping, approving, and receiving output

You may end elicitation explicitly by saying that there have been enough questions. The dialogue skill should then expose unresolved high-priority matters and ask whether to resolve, defer, or remove them. Deferred questions become stated limitations. Ending the dialogue is not equivalent to approving the eventual document.

Review the synthesized understanding and unresolved limitations before approving `kstar.md`. Then review the outline and planned figures, tables, and algorithms before approving `structure.md`. Approve the actual content rather than only a count of entries. The instructions ask Claude to record these decisions through the confirmation commands.

During production, request revisions if terminology, scope, or emphasis has drifted. The final PDF should be accompanied by a clear account of any remaining material limitations. Request the LaTeX source directory if you intend to edit, archive, or publish the document. A full session backup also preserves the ledger and decisions that led to it.

3.5 Continuing and revising

To continue, reopen a Cowork task with access to the same session folder and identify the session. The plugin can reload the current phase, ledger, and recent dialogue summaries. It cannot reconstruct material that was never saved. Where several sessions exist, provide the session name or path to avoid ambiguity.

Changes to knowledge should first update the ledger, then regenerate and reapprove the consensus, and revise the outline if affected. Wording-only changes can be made in LaTeX followed by rebuilding and review. A profile change requires synchronizing the session setting and document class options, because changing one does not automatically rewrite every existing file.

Implementation reference skills/sked-start/SKILL.md;skills/sked-dialogue/SKILL.md;skills/sked-latex/SKILL.md.

4 Component architecture

4.1 Layered organization

Figure 1 shows the principal dependencies. The human and Cowork interaction layer invokes phase skills. Skills coordinate file operations and the Python engine. The rendering path uses a separate build utility, bibliography formatter, report class, and external typesetting tools. The reviewer produces a structured assessment consumed by the quality calculation.

4.2 Skills and reviewer

Component	Phase	Responsibility
`sked-start`	1 and routing	Suitability, session creation, context packet, initial frame, and continuation routing.
`sked-dialogue`	2	Question selection, interpretation, ledger operations, round closure, and stopping decisions.
`sked-synthesis`	3	Resolve proposals and conflicts, verify sources, write and obtain approval of the consensus.
`sked-structure`	4	Plan sections and artifacts, associate entries with sections, check coverage, and obtain approval.
`sked-latex`	5	Scaffold, write sources, build, inspect pages, invoke quality assessment, and deliver.
`sked-quality`	5	Check coverage, coordinate coherence review, judge formal criteria, and compute acceptance.
`sked-reviewer`	Review role	Identify contradictions, unsupported content, terminology problems, and logical gaps against the PDF and consensus.

The reviewer agent is instructed to use fresh context and not receive the dialogue. This provides a

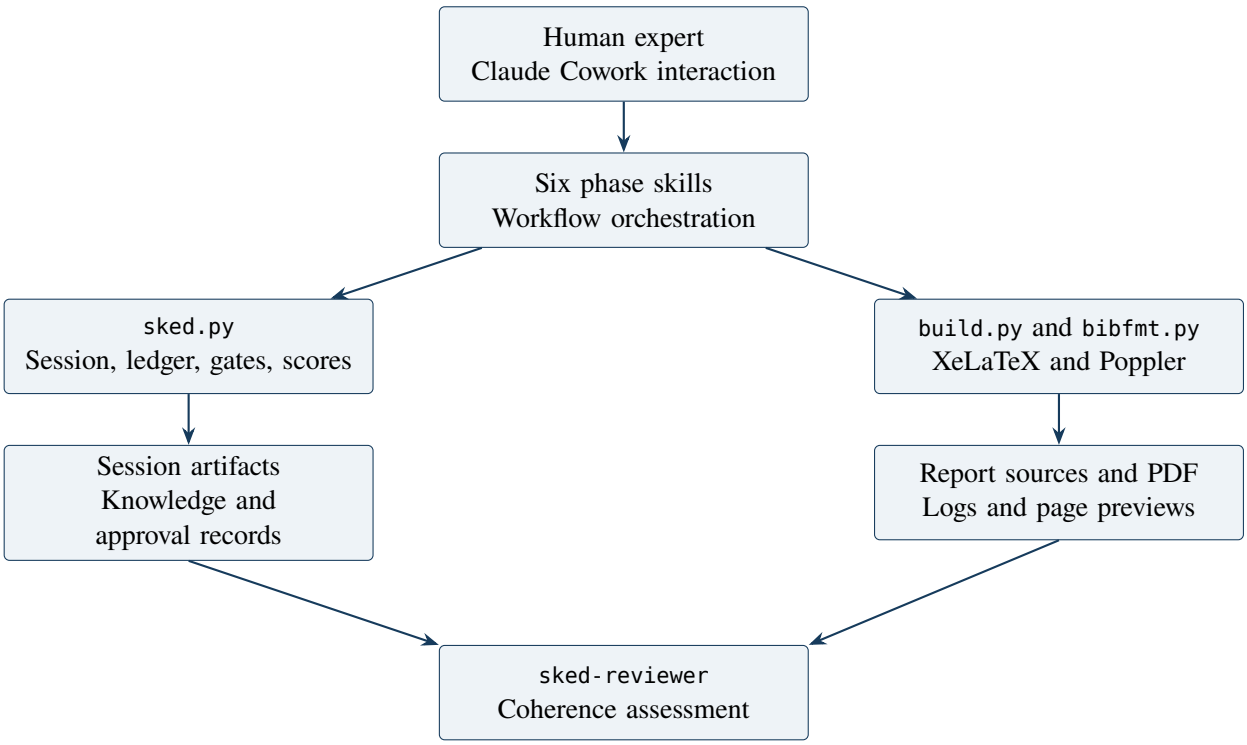


Figure 1: Principal SKED components and artifact dependencies. The reviewer reads the PDF and approved knowledge, without the dialogue under the prescribed review procedure.

different reading task, but it is not an external human review or proof of statistical independence. If a separate agent cannot be used, the quality skill prescribes self-review and disclosure that the review was not independent. The reviewer should report issues without rewriting the document.

4.3 Executable utilities

The session engine `sked.py` owns session creation, event operations, state reconstruction, convergence, exports, file confirmations, coverage, scaffolding, and quality scoring. It uses the Python standard library. Its `Session` class reads the session metadata and logs, and writes metadata through a temporary file followed by replacement.

The build utility `build.py` checks or installs selected dependencies when requested, renders the bibliography, invokes XeLaTeX, analyzes logs, examines PDF metadata and fonts, and generates page previews. It imports `bibfmt.py`, which parses BibTeX-like input and renders references for the selected profile. Optional Pillow support produces contact sheets from the individual page images.

The class `skedreport.cls` defines report typography, headings, localized labels, title material, captions, citations, appendix behavior, and invisible ledger tags. The template `main.tex` is populated with metadata during scaffolding. The documentation itself is a separate self-contained LaTeX file and does not require this custom class.

4.4 Execution and persistence boundary

Claude orchestrates the work. The scripts do not autonomously ask questions, call a model API, search the web, or decide whether an expert has genuinely consented. Commands can be called directly outside the intended dialogue. Accordingly, phase checks constrain script-mediated transitions but are not a complete authorization or policy-enforcement boundary.

State is stored as ordinary files in the selected workspace. Preservation across Cowork tasks depends on retaining that folder and making it accessible again. Plugin installation and session persistence

are separate concerns. Updating the installed plugin does not by itself migrate session data or prove compatibility with older generated class files.

Implementation reference scripts/sked.py;scripts/build.py;scripts/bibfmt.py;agents/sked-reviewer.md.

5 Session artifacts and knowledge representation

5.1 Directory structure

```
sked-sessions/<slug>/
  session.json
  ledger.jsonl
  dialogue.jsonl
  kstar.md
  structure.md
  report/
    main.tex
    skedreport.cls
    refs.bib
    references.tex
    sections/
    figures/
    main.pdf
    build/
      build-summary.json
      preview/
  quality/
    coherence.json
    fc-judged.json
    quality.json
    quality.md
```

The session slug is derived from the topic or supplied explicitly. Initialization refuses to replace an existing `session.json` at the selected location. Metadata includes the plugin version, topic, timestamps, phase, next round, mode, languages, parameters, context, metrics, approvals, and recorded convergence.

`ledger.jsonl` stores knowledge events. `dialogue.jsonl` stores compact exchanges and move labels. The latter is not guaranteed to be a verbatim transcript. The consensus and outline are human-readable Markdown artifacts. The report directory contains both editable sources and derived output. Quality artifacts preserve the structured assessments used by the score.

5.2 Entries and provenance

Entry identifiers use `K` for knowledge, `Q` for questions, and `S` for sources, followed by an integer. The knowledge types are claim, definition, assumption, constraint, decision, requirement, and example. Questions and sources are additional entry types.

Field	Meaning and supported values
`id`, `type`	Stable entry identifier and one of the supported categories.
`text`	An atomic statement or question, normally in the document language.

Field	Meaning and supported values
`origin`	`H` for human, `M` for model, or `S` for external-source content.
`status`	`proposed`, `agreed`, `contested`, `open`, `deferred`, `resolved`, or `retracted`.
`priority`	`high`, `normal`, or `low`. Coverage weights are 3, 2, and 1.
`verified`	`none`, `human`, or `source`. This is a recorded assessment.
`refs`	Links to source identifiers associated with an entry.
`term`, `note`	Definition term and supplementary explanation.
`bib`, `bibkey`	Bibliographic input and citation key for a source entry.
`section`	An editable section association supported by the engine.

For an ordinary new human-origin entry, the default status is agreed and verification is human. Model-origin contributions default to proposed and unverified. Questions default to open. Assigning agreed status to an unverified model entry marks it human-verified during replay. Source verification still requires an explicit source-level assessment.

The engine validates enumerated values on several operations, but it is not a comprehensive schema validator. Status values are not restricted to a type-specific transition graph. Some edit fields receive less validation than initial additions. Source links are not automatically dereferenced or evaluated for evidential support. Operational discipline remains necessary.

5.3 Event operations and replay

The supported operations are `add`, `edit`, `status`, `verify`, `retract`, and `turn`. Knowledge operations are appended to the ledger with timestamps and round numbers. Turn operations go into the dialogue log. Replay reconstructs entries in event order and retains revision information. Retracted entries are omitted from ordinary views but can be requested explicitly.

A batch can refer to an earlier addition through `$1`, `$2`, and similar placeholders. These count additions, not arbitrary operation positions. Forward references should not be used. The engine collects new events before appending them, but writes the ledger, dialogue, and metadata separately. There is no multi-file database transaction or session lock. Run one writer per session and preserve backups before manual recovery.

5.4 Consensus export

The exporter collects agreed or resolved knowledge entries, groups definitions and statements, lists open and deferred questions, and includes source records. It preserves a narrative between designated markers when regenerating the file. The skill asks Claude to write that narrative in approximately 120–300 words, using only ledger content.

The actual export may also list outstanding proposed or contested entries under an explicit exclusion heading. The confirmation command normally refuses outstanding non-source, non-question proposals or conflicts and rejects an unchanged narrative placeholder. It does not prove that the narrative is faithful or that all questions were resolved. Review the complete artifact before approval.

Implementation reference references/ledger-schema.md;scripts/sked.py,Session,apply_ops,fold,build_kstar.

6 Five phases and transition checks

6.1 Initialization

Phase 1 creates the session and context, constructs the initial model frame $K^{(0)}$, and records the first question. Round zero is initialization rather than an elicitation round. Closing it records an initial

metric and increments the next-round counter. Later phase transitions check that a metric for round zero exists.

A useful initial frame makes scope, assumptions, and unknowns visible. It must remain challengeable. The mode and context packet are stored configuration; the quality of the frame and the suitability of the first question are prescribed by the skills rather than computed by the engine.

6.2 Socratic dialogue

Phase 2 iterates through interpretation, knowledge extraction, next-move selection, event application, and round closure. Each answer can add, refine, contest, resolve, defer, or retract entries. Counterquestions from the human should be answered before Claude asks the next elicitation question.

Round closure records metrics and reports convergence readiness, budget exhaustion, and the model-proposal acceptance warning. It does not itself move to synthesis. The `converge` command records why elicitation ended, after which the phase command can enter phase 3. A numerical convergence request is rejected when criteria fail unless overridden. Human and budget stopping modes can record unresolved reasons.

6.3 Synthesis

Phase 3 resolves pending model proposals and contested knowledge, evaluates remaining questions, and verifies factual claims and bibliography records as prescribed. Unanswered model proposals should not be silently accepted. An unresolved question can be deferred, but its limitation must remain visible.

Claude writes the consensus narrative, shows it to the expert, and records approval only after the expert agrees. Confirmation saves the file hash, timestamp, and round. There is no separate authenticated human signature. A later phase check validates file identity against this saved hash.

6.4 Structuring

Phase 4 turns the approved knowledge into an outline and plans informative artifacts. The skill prescribes one primary section for each target entry and full plan coverage. Entries that have no meaningful section should trigger reconsideration of the outline or scope.

The plan coverage utility recognizes entry identifiers appearing anywhere in the Markdown file. It does not parse primary-section assignments, prove uniqueness, or determine whether a section adequately represents an entry. Moreover, `confirm structure` does not automatically run coverage. Full mapping is therefore a prescribed step that must be performed and reviewed before confirmation.

6.5 Production and acceptance

Phase 5 scaffolds the report, generates sections from the approved knowledge and outline, produces planned artifacts, compiles, inspects pages, and assesses quality. New knowledge discovered during writing should return to the ledger and approval workflow. It should not be inserted directly as an untracked factual addition.

The transition to `done` checks prior approvals and the stored quality result. The intended workflow rebuilds and recomputes that result immediately before delivery. The engine does not automatically establish that the stored result is newer than every edited source file.

6.6 What transition checks guarantee

Requested target	Normal programmatic prerequisites
Phase 2	A closed round zero.
Phase 3	Closed round zero and a recorded convergence decision.
Phase 4	The preceding conditions and an existing consensus file matching its confirmation hash.
Phase 5	The preceding conditions and an existing outline matching its confirmation hash.
`done`	The preceding conditions and a stored quality result with `passed` set to true.

These checks are based on the requested target. The engine does not require adjacent transitions and does not block every command that would be inappropriate in the current phase. The `--force` option bypasses checks where provided. The skills reserve it for an explicit human override, but that restriction is procedural. Avoid it during ordinary use.

Implementation reference scripts/sked.py,cmd_round_close,cmd_converge,cmd_confirm,cmd_phase.

7 Convergence and dialogue indicators

7.1 The state projection

For an active ledger state, the implementation constructs

$$A_t = \{(\mathrm{id}_i, \mathrm{norm}(\mathrm{text}_i), \mathrm{status}_i) \mid i \in \mathcal{E}_t\}. \tag{3}$$

The symbols and set notation have the following meanings.

- A_t is the set of projected active-entry tuples after all events through round t have been replayed.
- t is the nonnegative integer round index. Round 0 represents initialization.
- $\mathcal{E}_t$ is the index set of all active ledger entries at round t, excluding retracted entries.
- i identifies one entry in $\mathcal{E}_t$; id_i is its stable ledger identifier, such as `K1`, `Q1`, or `S1`.
- text_i is the stored text of entry i at the selected round, and status_i is its status at that round. The round dependence of these fields is implicit to match the implementation-oriented notation.
- $\mathrm{norm}(\cdot)$ is the text-normalization function described below; the centered dot denotes its argument.
- Parentheses delimit an ordered tuple. Braces collect the tuples into a set. The vertical bar means "such that", and $i \in \mathcal{E}_t$ means that the tuples are formed for entries in the active index set.

Normalization applies Unicode NFC normalization, case folding, whitespace collapse, and trimming. Retracted entries are excluded. The resulting set includes active questions and sources as well as knowledge entries. Changes to priority, verification, references, or other metadata do not change this projection when text and status remain unchanged.

The round distance is

$$\delta_t = 1 - \frac{|A_t \cap A_{t-1}|}{|A_t \cup A_{t-1}|}, \tag{4}$$

The symbols and operators have the following meanings.

- δ_t is the dimensionless Jaccard distance between the projected ledger states at rounds t and $t-1$.
- t is the current closed-round index, and $t-1$ identifies the preceding round. For initialization, the implementation compares round 0 with an empty prior state.
- A_t and A_{t-1} are the current and preceding sets of active-entry tuples, respectively.
- $\cap$ denotes set intersection, so $A_t \cap A_{t-1}$ contains tuples unchanged between the two states.
- $\cup$ denotes set union, so $A_t \cup A_{t-1}$ contains every distinct tuple present in either state.

- $|B|$ denotes the cardinality, or number of elements, of a set B; B is a generic set used here to explain the notation. The constant 1 converts Jaccard similarity into distance.

The implementation returns $\delta_t = 0$ when both sets are empty, avoiding a zero denominator. The distance lies between zero and one on the projected sets. It is not a distance that distinguishes all possible full ledger states, because different metadata can map to the same projection.

7.2 Interpretation and an example

If ten entry tuples remain unchanged and one is added, the distance is $1/11$, approximately 0.091. If the text of one among ten entries is replaced, the union contains eleven tuples and the intersection nine, giving approximately 0.182. The indicator treats the replacement as removal of an old tuple and addition of a new one. It does not measure semantic similarity between the two formulations.

A priority-only change can yield zero distance. Conversely, a small textual revision may create a substantial distance in a short ledger. A large stable ledger can dilute the effect of a new important entry. These consequences follow from the chosen projection and denominator. They should guide interpretation of the dashboard rather than be hidden behind the word convergence.

7.3 Default stopping conditions

Automatic readiness requires at least three closed dialogue rounds excluding round zero, two consecutive stored distances strictly below 0.05, no open high-priority question, and no contested entry. The engine stores the distances rounded to four decimal places, and readiness uses those stored values. The maximum-round budget is fifteen dialogue rounds by default. Reaching it asks for a human decision under the skill policy rather than automatically proving agreement.

Pending model proposals are reported separately and do not directly block the numerical readiness calculation. They must still be resolved during synthesis before ordinary consensus confirmation. Open questions of normal or low priority may also remain. The human's decision can end elicitation even if the numerical conditions are not satisfied, provided that the unresolved matters are handled explicitly.

7.4 Model-proposal acceptance warning

The engine counts model-origin non-source, non-question entries whose final status is agreed, retracted, or contested. It reports how many were agreed without an edit. A ratio above 0.8 across at least five such decisions triggers a warning. The prescribed response is to ask more open questions and invite the expert to restate important positions.

This indicator is a prompt to reconsider the dialogue. It does not diagnose sycophancy or determine whether the accepted proposals were correct. A knowledgeable expert can legitimately accept several correct proposals unchanged. The warning is not a hard transition blocker.

Implementation reference scripts/sked.py,norm,state_set,jaccard_distance,convergence,m_acceptance.

8 Document quality and acceptance

8.1 Composite score

The configurable score and its default coefficients are

$$Q(R) = \alpha C_q + \beta C_{\text{oh}} + \gamma F_c = 0.4 C_q + 0.3 C_{\text{oh}} + 0.3 F_c, \tag{5}$$

The symbols and coefficients have the following meanings.

- R is the document being assessed, and $Q(R)$ is its dimensionless composite acceptance score.

- C_q is the dimensionless, priority-weighted coverage of target knowledge entries. The subscript q is the package's coverage-component label, not a summation index.
- C_{oh} is the dimensionless coherence score derived from the review. The subscript oh is a descriptive component label; together with C it corresponds to the package's `Coh` notation.
- F_c is the dimensionless formal-compliance score, defined as the fraction of included checks that pass. The subscript c labels compliance and is not the critical-issue index used later in n_c.
- α, β, and γ are dimensionless weights assigned to coverage, coherence, and formal compliance, respectively. Their defaults are 0.4, 0.3, and 0.3; their intended constraints are nonnegativity and $\alpha + \beta + \gamma = 1$.

The default acceptance threshold is 0.85. With component values in $[0, 1]$ and the intended weight constraints, $Q(R)$ also lies in $[0, 1]$. The score is an internal workflow criterion. No calibration study, measurement uncertainty, inter-rater reliability estimate, or comparison benchmark is provided in the archive.

Weights and thresholds are configurable. The setter accepts many values without comprehensive numerical validation. Keep nonnegative weights summing to one if the score is to retain its intended interpretation. A higher threshold does not repair weak evidence or incomplete checklists.

8.2 Tagged knowledge coverage

The target set contains agreed or resolved knowledge entries and deferred questions. Each target has weight three, two, or one for high, normal, or low priority. Coverage is

$$C_q = \frac{\sum_{i \in \mathcal{T}} w_i I_i}{\sum_{i \in \mathcal{T}} w_i}, \tag{6}$$

The symbols and operators have the following meanings.

- C_q is the dimensionless weighted coverage score for the outline or report under inspection.
- $\mathcal{T}$ is the target set of agreed or resolved knowledge entries and deferred questions that should be represented.
- i indexes a target entry, and $i \in \mathcal{T}$ indicates that the sums range over that target set.
- w_i is the positive, dimensionless priority weight of entry i, equal to 3, 2, or 1 for high, normal, or low priority, respectively.
- I_i is a binary coverage indicator. It equals 1 when the entry identifier is detected in the inspected outline or report tags and equals 0 otherwise.
- $\sum$ denotes summation over the indicated target entries. The numerator is the total covered weight, and the denominator is the total target weight.

For a nonempty target set, C_q lies in $[0, 1]$. An empty target set produces coverage one by implementation convention. This is a degenerate condition, not evidence of a complete technical report.

For an outline, identifiers are detected in Markdown. For a report, the engine scans LaTeX files recursively, excluding directories named `build` and files named `references.tex`, and extracts `\sked{...}` tags. The tags are invisible in the ordinary PDF. A review build exposes them for inspection.

The scan is lexical. It is not restricted to the exact transitive input graph of the compiled document, so an unused LaTeX file in the report tree can contribute tags. A tag does not prove that surrounding prose expresses the entry. Untagged claims can escape coverage accounting entirely. The quality skill therefore requests spot checks, and the reviewer compares document content with the approved knowledge.

8.3 Coherence assessment

The reviewer examines contradictions, distortion of the consensus, unsupported claims, inconsistent terminology, logical gaps, unsupported abstracts or conclusions, and deferred matters presented as settled. Issues are classified as critical, major, or minor. The default calculation is

$$C_{\text{oh}} = \max(0, 1 - 0.15n_c - 0.07n_m - 0.02n_l), \tag{7}$$

The symbols, constants, and operator have the following meanings.

- C_{oh} is the dimensionless coherence score for the assessed document.
- n_c is the nonnegative integer count of critical issues; the subscript c denotes critical severity.
- n_m is the nonnegative integer count of major issues; the subscript m denotes major severity.
- n_l is the nonnegative integer count of minor issues; the subscript l denotes minor severity.
- 0.15, 0.07, and 0.02 are the dimensionless penalties per critical, major, and minor issue, respectively.
- The constant 1 is the score assigned when no issues are recorded. The constant 0 is the lower bound.
- $\max(a, b)$ returns the larger of its two arguments a and b; these letters are generic operator arguments, not additional document parameters. Here the operator prevents accumulated penalties from producing a negative score.

Under this penalty rule, C_{oh} lies in $[0, 1]$. The assessment is stored in `quality/coherence.json`. Missing review is a blocker.

The reviewer instructions say to omit a numerical score because the engine computes it. The implementation nevertheless accepts an explicit `score` field if one is supplied. For reproducible use of the documented penalty rule, omit that field and preserve the issue records. Coherence depends on reviewer sensitivity and issue granularity, so scores are not directly comparable across reviewers without a controlled protocol.

8.4 Formal compliance

Formal checks combine automatic records and optional human- or model-judged items. Automatic checks inspect compilation, undefined references and citations, excessive overfull boxes, missing glyphs, hyphenation, font embedding, bibliography consistency, source verification labels, float references, placeholders, selected style rules, and use of entries in inappropriate states. Judged items address title material, abstract, keywords, section order, captions, terminology, page layout, limitations, and profile-specific requirements.

The formal component is the count of passed items divided by the number of included items. Checks have equal weight. Adding or omitting judged items changes both the numerator and denominator. The scoring function does not require a complete judged checklist to exist. Follow the profile checklist and retain every result rather than selecting only easy passes.

Several checks are heuristics. Source verification is based on stored labels, not a fresh source retrieval. The numeric-claim detector looks for a digit in a tagged model-origin entry and does not understand the meaning of the number. Static reference and style checks use regular expressions rather than a complete TeX parser.

8.5 Hard blockers and procedural failures

The implemented hard blockers are an absent or unsuccessful build, uncovered high-priority targets, tagged model-origin claims containing digits without source verification, critical coherence issues, missing coherence review, and cited keys missing from the bibliography build result. Acceptance requires meeting the score threshold with none of these blockers.

Other failed formal items can coexist with a passing composite score. Thus a programmatic pass is weaker than the skill's prescribed procedure, which asks Claude to fix failed checks and inspect rendered pages. The build utility's successful exit also means less than final acceptance. It indicates production of a PDF without recorded LaTeX errors, not satisfaction of every layout or factual requirement.

The quality instructions allow at most three repair loops by default before remaining issues are presented to the user. This limit is stored as a parameter but is not an autonomous loop enforced by the scoring function. The model follows it as a workflow instruction.

Implementation reference scripts/sked.py,coverage,static_tex_checks,quality;skills/sked-quality/SKILL.md.

9 LaTeX production and bibliography pipeline

9.1 Scaffolding and content generation

Scaffolding creates the report, sections, and figures directories, copies the report class, renders the initial metadata into `main.tex`, and writes bibliography input from ledger source entries. Existing `main.tex` is retained unless forced. The class is copied and the bibliography is regenerated during scaffolding, so back up intentional local modifications before repeating it.

Claude writes one source file per major section and adds the relevant input statements. Knowledge-bearing paragraphs, lists, captions, and table notes should carry ledger tags. New claims require ledger handling and approval. Abstract and conclusions should summarize supported content rather than introduce additional results.

The class supports metadata commands, localized cross-references, figures, tables, algorithms, Unicode-safe code listings, and appendices. Figure labels and arrow routes should be checked visually. For Cyrillic code, the package prescribes its `skedcode` environment rather than relying on ASCII-oriented listing behavior under XeTeX.

9.2 Profiles and their scope

Profile	Implemented intent and main defaults
`ieee`	A technical-report layout with numeric references, A4, 11 pt text, side margins of 25 mm, and top and bottom margins of 27 mm. It is not an official journal-specific IEEE template.
`dstu`	A Ukrainian research-report layout with 14 pt text, 1.5 spacing, and left/right/top/bottom margins of 25/15/20/20 mm. References use a simplified DSTU-oriented formatter. Institutional requirements still need checking.
`apa`	An APA-oriented report layout with A4 paper, 12 pt text, double spacing, 25.4 mm margins, and author-year references. Exact publication compliance requires review.

The engine localizes its generated consensus text for Ukrainian and English. The class and build utility recognize additional language names in selected places, but their presence does not establish complete localization or tested multilingual support. Fonts are selected from an availability-ordered list that prefers Times New Roman and then alternatives including Liberation Serif.

9.3 Bibliography rendering

The Python formatter reads `refs.bib`, follows supported citation and input patterns, orders references, creates `references.tex`, and reports missing keys, uncited entries, and incomplete records. Numeric profiles default to citation order; the APA path uses author-oriented ordering. An explicit bibliography order can also be requested.

This removes the normal dependency on an external BibTeX or biber run for this pipeline. It is a custom parser and renderer rather than a complete implementation of every BibTeX construct or every style rule. Verify names, transliteration, punctuation, dates, URLs, and the rendered entries against the target venue. A plausible reference format does not verify that a publication exists or supports the cited statement.

The package README suggests that delivered sources compile anywhere with two XeLaTeX runs. The executable build path generates `references.tex` first and may perform additional runs for settled references. Portable delivery therefore requires the generated bibliography file or access to the formatter, the report class, appropriate fonts, and required TeX packages.

9.4 Build and inspection

The build command optionally checks dependencies, renders references, invokes XeLaTeX at least twice under its normal default run limit, and repeats when the log indicates another run is needed, up to the configured maximum. It records errors, unresolved references, citations, overfull boxes, glyph issues, font warnings, hyphenation information, and PDF metadata in `build-summary.json`.

Poppler tools supply page information, font checks, and PNG previews. Pillow can assemble contact sheets. Generating previews does not mean that they were viewed. The skill explicitly requests inspection of all pages through thumbnails and full-size inspection of the title page and pages containing complex artifacts.

The `--ensure-deps` option can invoke `apt-get` when the process is root and that package manager is available. This is an environment-modifying operation, not a portable installation mechanism. It may be unavailable in Cowork or constrained by package repositories and network policy. Missing dependencies should be reported accurately rather than hidden behind repeated build attempts.

Implementation reference scripts/build.py;scripts/bibfmt.py;templates/skedreport.cls;references/latex-guide.md.

10 Command and configuration reference

10.1 Execution convention

Ordinary users interact with Claude in natural language. The commands below are for maintainers, diagnosis, and explaining what Claude executes. Replace the plugin path and session directory with actual paths in the Cowork execution environment. The examples use shell variables only as readable local aliases.

```
SKED_PLUGIN_DIR=/actual/path/to/sked
SKED_SESSION_DIR=/actual/work/sked-sessions/example
python3 "$SKED_PLUGIN_DIR/scripts/sked.py" \
  --session "$SKED_SESSION_DIR" status
```

The global `--session` option precedes the subcommand. Session discovery checks the explicit argument, then `SKED_SESSION`, then the current directory, then a single session beneath `./sked-sessions`. Multiple matches require an explicit choice.

10.2 Session commands

Command	Purpose and important options
`init`	Requires `--topic`. Options include `--root`, `--slug`, `--profile`, `--mode`, `--lang-dialogue`, and `--lang-doc`. Defaults are IEEE, deep mode, and Ukrainian languages.
`list`	Lists sessions beneath `--root`.
`set KEY VALUE`	Sets a dotted metadata path. Attempts JSON parsing of the value and otherwise stores a string.
`apply FILE`	Applies event operations from a file, or from standard input when the file argument is a hyphen.
`round-close`	Stores current-round metrics and advances the round. Supports `--note` and `--json`.
`show`	Filters entries by type, status, origin, identifier, or round. `--all` includes retracted entries. Output can be Markdown or JSON.
`dialogue`	Shows saved exchanges, optionally limited with `--last`. Supports JSON output.
`status`	Displays phase, counts, metrics, unresolved conditions, approvals, and the next suggested step.
`converge`	Requires `--by delta`, `human`, or `budget`. Records stopping reasons and an optional note.
`phase N`	Requests phase 1–5 or `done`, subject to target prerequisites unless forced.
`kstar`	Exports the consensus file and preserves its narrative block. Supports an alternate output path.
`confirm kstar`	Records the consensus file hash after normal proposal and placeholder checks.
`confirm structure`	Records the outline file hash. Does not itself validate plan coverage.
`coverage`	Scans the report or a file supplied through `--plan`. Supports JSON output.
`scaffold`	Creates report sources and writes class and bibliography files. `--force` can replace the main file.
`bib`	Rewrites bibliography input from ledger source entries.
`quality`	Computes and writes the composite assessment for the selected report directory.

10.3 Default parameters

Parameter	Default	Meaning
`epsilon`	0.05	Strict upper bound for stable round distances.
`stable_rounds`	2	Required consecutive stable rounds.
`min_rounds`	3	Minimum closed dialogue rounds.
`max_rounds`	15	Budget notification point.
`weights.alpha`	0.4	Tagged knowledge coverage weight.
`weights.beta`	0.3	Coherence weight.
`weights.gamma`	0.3	Formal compliance weight.

Parameter	Default	Meaning
`q_threshold`	0.85	Composite acceptance threshold.
`overfull_tolerance_pt`	1.0	Quality-check tolerance for reported overfull boxes.
`max_fix_loops`	3	Prescribed maximum repair loops before reporting remaining issues.

These values are stored under `params`. For example, changing `params.epsilon` changes the intended convergence threshold. Such changes should reflect the task and be documented before assessment, rather than made only to obtain a pass. Context and house-style rules are separate fields under `context`.

10.4 Build commands

```
python3 "$SKED_PLUGIN_DIR/scripts/build.py" \
  --dir "$SKED_SESSION_DIR/report"

python3 "$SKED_PLUGIN_DIR/scripts/build.py" \
  --dir "$SKED_SESSION_DIR/report" --review

python3 "$SKED_PLUGIN_DIR/scripts/sked.py" \
  --session "$SKED_SESSION_DIR" coverage --json

python3 "$SKED_PLUGIN_DIR/scripts/sked.py" \
  --session "$SKED_SESSION_DIR" quality
```

Additional build options include `--ensure-deps`, `--no-previews`, `--dpi`, `--max-runs`, `--bib-order`, `--main`, and `--update-class`. Replacing a customized class with `--update-class` should be intentional. The standalone bibliography utility also exposes command help for formatter-specific use.

11 Illustrative workflow

11.1 Scenario and initialization

This example is a recommended teaching scenario, not a measured Cowork case study. Suppose a research engineer wants to document an internal dataset-cleaning procedure. The desired output is an English report that another engineer can reproduce. The expert knows the processing steps but has not yet separated requirements, assumptions, and observed outcomes.

Claude collects the intended reader, example inputs, available measurements, and exclusions. The initial model frame proposes that the report should cover input validation, duplicate handling, missing-value policy, output artifacts, and limitations. Those are proposed areas for discussion rather than claims that the procedure actually implements them.

11.2 Elicitation and ledger changes

The expert explains that duplicate records are detected by a normalized identifier, while near-duplicates are outside scope. Claude should record the first statement as a claim and the second as a constraint. If the expert says the procedure is probably faster than an earlier version, the statement remains a proposal or assumption until supporting evidence exists.

A minimal illustrative event batch might be

```
[
  {"op":"add", "type":"claim", "origin":"H",
   "text":"Exact duplicates are detected by normalized identifier.",
   "priority":"high"},
  {"op":"add", "type":"constraint", "origin":"H",
   "text":"Near-duplicate detection is outside the procedure scope."},
  {"op":"add", "type":"question", "origin":"M",
   "text":"What evidence supports a performance improvement?",
   "priority":"normal"}
]
```

The human-origin defaults record agreement and human verification. That is appropriate for describing the expert's own procedure, but it does not independently establish implementation correctness. If the performance question cannot be answered, it can be deferred and later presented as a limitation.

11.3 Consensus and structure

After the dialogue is ended and recorded, Claude resolves outstanding proposals, exports the consensus, and writes a short narrative that preserves the distinction between supported behavior and unmeasured performance. The expert approves the content. The outline then assigns each entry to a primary section and plans a process diagram, a table of input assumptions, and a reproducibility checklist.

A sound coverage review asks whether each section actually explains its associated entries. Merely putting every identifier into a list would satisfy the lexical plan scan but defeat its purpose. Approval should follow this substantive review.

11.4 Production and delivery

Claude writes section files, marks knowledge-bearing material with ledger tags, and creates the diagram and tables. The build produces a PDF and diagnostics. The reviewer checks the PDF against the consensus, including whether the abstract accidentally claims a measured speed improvement. Formal review checks captions, page layout, citations, and stated limitations.

The final delivery includes the compiled report and, when requested, the source files. The limitation concerning performance remains explicit. No benchmark numbers or quality scores are supplied in this example because none were measured. A later experiment should enter through new ledger evidence, followed by the relevant approvals and rebuild.

12 Limitations and assurance boundary

12.1 Knowledge and model limitations

SKED cannot guarantee that an expert's statement is correct, that a model interpretation preserves every nuance, or that a retrieved source supports a claim. The ledger improves inspectability only when entries are atomic, attributed correctly, and reviewed. Incorrect agreement can be retained faithfully. A stable dialogue can also be incomplete if important questions were never asked.

Source verification is represented by a label and references. The engine does not automatically inspect remote evidence, reproduce experiments, or authenticate the reviewer. Its digit detector can miss quantities written as words and can flag an identifier or version that is not an empirical claim. Human and model responsibilities therefore remain central to factual reliability.

12.2 Approval and state consistency

File hashes detect changes to the approved files. They do not automatically detect changes to the live ledger after approval. Coverage is calculated from the live ledger, while generation is prescribed to use the approved consensus. If the ledger changes without regenerating that file, these bases can diverge. The required operational response is to regenerate and reapprove affected artifacts.

There is no complete state machine restricting every command to its intended phase. Nonadjacent phase requests are possible when their prerequisites exist. Confirmation and override commands are callable by the same process that writes the files. This design supports an assisted workflow, not an independently authenticated approval system.

12.3 Coverage and quality limitations

Tagged coverage can be inflated by irrelevant tags or unused source files. Untagged additions can escape the accounting. Plan coverage does not establish one-to-one assignment or semantic adequacy. Reviewer findings depend on the reviewer, and a numerically high score can coexist with failed nonblocking checks.

Quality output and build summaries have no comprehensive freshness linkage to all inputs. The transition to done reads a stored pass. Always rebuild, review, and recalculate after relevant changes. The score should not be presented as a publication-readiness probability, a truth score, or a validated scientific measure.

12.4 Persistence, compatibility, and portability

Session files have no database-level concurrency protection or transactional update across all artifacts. Direct edits to the ledger can destroy replay validity. No explicit migration mechanism is included for future schema changes. Keep the inspected plugin version with archived sessions where reproducibility matters.

The README specifies Python 3.8 or later. The targeted execution checks for this manual used Python 3.12.14, not a supported-version matrix. Compatibility with every advertised Python version has not been established by those checks. Test the actual deployment interpreter and its dependencies before production use.

Cowork compatibility depends on installation, file access, Python execution, tool availability, and typesetting dependencies. No complete installation or interactive session was run in Cowork for this edition. Host operating-system paths should not be confused with paths inside the execution environment. A profile name does not establish exact compliance with a journal's submission template.

12.5 Data handling and scope of protection

The ledger, dialogue summaries, and reports may contain the expert's substantive knowledge. The package provides no encryption, access-control layer, secret redaction, or retention policy for these artifacts. Apply the host environment's file permissions and the organization's data-handling requirements. Local file storage alone does not imply that interactions with a hosted model remain entirely local.

The scripts and templates do not constitute a hardened sandbox for untrusted LaTeX or arbitrary external files. The appropriate operational boundary is a controlled working folder with reviewed source material. This statement describes absent mechanisms rather than an evaluated security vulnerability.

Implementation reference scripts/sked.py;scripts/build.py;README.md;references/sked-method.md.

13 Troubleshooting and recovery

Observed condition	Diagnosis and recommended action
Plugin is installed but no session starts	Confirm that SKED is available in the current Cowork task, that its resources can be located, and that the task can write to the chosen folder. Use an explicit start request.
Several sessions are reported	Identify the intended session and use its explicit directory. Do not let a new task silently select a different project.
Transition to synthesis fails	Inspect round zero and the convergence record. Numerical readiness must be explicitly recorded through the stopping command.
Consensus confirmation fails	Resolve pending proposals or contested entries and replace the narrative placeholder. Do not force approval merely to proceed.
Approved file changed	Review the new content and confirm it again. A new hash is expected after an intentional change.
Ledger changed after approval	Regenerate the consensus, review and approve it, and revise the outline and report where needed. File-hash checks alone do not perform this reconciliation.
XeLaTeX or packages missing	Check the execution environment. Use dependency installation only where supported and permitted. Report missing components when installation is unavailable.
Undefined citation or missing bibliography key	Correct the ledger source record and citation key, regenerate bibliography input, and rebuild. Do not edit only the derived reference file.
Overfull boxes or missing glyphs	Inspect the reported location and rendered page. Reword long tokens, adjust tables or diagrams, and choose fonts with the required characters.
Quality result is stale	Rebuild after edits, repeat relevant page inspection and review, then recalculate quality before marking the session done.
Malformed ledger JSON	Preserve a backup first. Locate the invalid line and restore from a known valid version. Avoid silently deleting history to obtain a successful replay.

A reliable archive contains the complete session directory, the exact plugin version, and the final report sources. For lightweight source delivery, omit temporary build files but retain `references.tex`, the report class, section files, bibliography input, and external figures. Record dependency versions if future reproduction is important. Changes to the build environment may alter line breaks, pagination, fonts, and warnings even when source text is unchanged.

14 Verification performed for this edition

14.1 Inspection scope

The documentation was prepared from the archive named on the title page. Its manifest, six skill instructions, reviewer instructions, five reference files, three Python scripts, and template interfaces were inspected. The executable source was used to qualify broader statements in the README, especially concerning gates, coverage, quality, and build portability.

The archive SHA-256 is given below to identify the inspected bytes.

```
8ba9cb85f453a283a550f16c922618e2998dcb04968b16ef8f41bf113f960ed6
```

This identifies a source snapshot. It does not authenticate its publisher or establish that a future repository download will contain the same bytes.

14.2 Targeted execution checks

Selected mechanisms were exercised in a temporary local session without a Cowork client. The checks support narrow implementation statements rather than an overall performance or compatibility claim.

Check	Observed result and interpretation
Priority-only ledger edit	The projected Jaccard distance was zero. Metadata changes are not represented by the state projection.
Outline containing only an entry ID	Reported plan coverage was 1.0 for a one-entry target set. Presence is checked lexically rather than semantically.
Two successive text edits	Replay returned the second text. Event order determines the reconstructed entry state.
Ledger edit after file approval	The unchanged consensus file still passed its saved hash check. Approval identity does not automatically bind to the current ledger.

The manual's LaTeX source is compiled and its rendered pages are checked separately as an artifact-production step. That confirms the readability of this manual, not the successful completion of a SKED-generated report. No full dialogue, independent document review, comparative experiment, journal compliance assessment, or end-to-end Cowork installation test is claimed.

14.3 Recommended acceptance exercise

Before relying on a new deployment, conduct one short Cowork session with a known subject and explicit expected statements. Confirm installation and resource access, create a session, record a correction, defer a question, approve knowledge and structure, generate the PDF, inspect a ledger tag, and verify that edits trigger the intended review procedure. Retain the files and actual observations.

For a research publication, separate this operational acceptance exercise from a controlled evaluation of SKED itself. Such an evaluation would require defined tasks, a baseline, an assessment protocol, independent judgments where feasible, and recorded outcomes. The archive does not supply those results, and this manual does not invent them.

A Source traceability

Package resource	Documentation basis
`.claude-plugin/plugin.json`	Identity, version, author, and declared purpose.
`README.md`	Workflow summary, operational definitions, profiles, dependencies, and claimed differences from the unavailable report.
`references/sked-method.md`	Conceptual model, Socratic components, context packet, phases, indicators, and stated limitations.

Package resource	Documentation basis
`references/ledger-schema.md`	Session files, fields, operations, and command summary.
`references/socratic-moves.md`	Question priorities, dialogue patterns, extraction rules, and anti-patterns.
`references/style-profiles.md`	Presentation profiles and judged formal checklist.
`references/latex-guide.md`	Class interface, ledger tags, artifacts, and rendering procedure.
`skills/*/SKILL.md`	Prescribed phase behavior, approvals, review, revisions, and delivery.
`agents/sked-reviewer.md`	Coherence-review scope, severity definitions, and issue format.
`scripts/sked.py`	Executable session behavior, metrics, gates, coverage, and quality.
`scripts/build.py`	Dependencies, compilation, diagnostics, and previews.
`scripts/bibfmt.py`	Bibliography parsing and rendering.
`templates/skedreport.cls`	Layout, localization, tags, captions, and report interfaces.
`templates/main.tex`	Scaffold substitutions and initial report structure.

B Reference material and document maintenance

1. Kharchenko, K. SKED plugin, version 0.1.0. Source package `Sked-0.1.0-v1.zip`. Repository consulted on 29 October 2026. https://github.com/konst1970/SKED.
2. Anthropic. *Use plugins in Claude*. Official Help Center. Consulted on 29 October 2026 for the general Cowork plugin interface and custom-file upload. https://support.claude.com/en/articles/13837440-use-plugins-in-claude.

The second source is used only for the brief host-interface guidance. All SKED-specific claims derive from the inspected archive. No independent scholarly bibliography was supplied with this documentation request. The unavailable SKED v1.0.0 technical report is mentioned only to explain a reference inside the package and is not treated as consulted evidence.

Future editions should update the archive identifier, plugin version, command behavior, client interface notes, observed compatibility results, and known limitations. If implementation changes invalidate a documented caveat, replace it with evidence from the new source and appropriate execution checks. Keep documentation versioning separate from plugin versioning.

C Terminology

Term	Meaning in this manual
Context packet	Stored audience, exemplars, constraints, rubric, and metadata used to guide a session.
Knowledge ledger	The append-only sequence of events from which the current knowledge entries are reconstructed.
Round	A recorded interaction unit whose closure stores metrics and advances the round counter.

Term	Meaning in this manual
Consensus K^*	The approved knowledge artifact used as the prescribed content basis for generation. Agreement does not imply truth.
Gate	A programmatic prerequisite check or, where explicitly stated, a prescribed human review step.
Coverage	Weighted presence of target identifiers in an outline or LaTeX tags.
Coherence	The review-based assessment of logical and terminological consistency and fidelity to the agreed knowledge.
Formal compliance	The fraction of included automatic and judged presentation checks that pass.
Profile	A bundled report layout and reference-rendering configuration.
Verification label	A stored value recording human or source assessment, rather than an automatic proof of correctness.